

## 7.14 Endlosschleifen

Endlosschleifen (Infinite loop) sind Schleifen, die nach jeder Abarbeitung erneut abgearbeitet werden, falls die Ausführung nicht durch äußere Einflüsse oder eine Anweisung aus dem Inneren (etwa die Sprunganweisung `break`) abgebrochen wird.

Endlosschleifen können durch Fehler entstehen, wenn die Abbruchbedingung nicht definiert ist oder nicht eintreten kann. Es gibt aber auch sinnvolle Anwendungen. So kann etwa die permanente Abfrage der Mausposition bei einer grafischen Oberfläche eine gewollte Endlosschleife sein. Eine andere Situation ist, dass man eine Schleife eine unbekannte oder unbestimmte Anzahl von Durchläufen ausführen möchte und der Abbruch dann durch eine Eingabe erfolgen soll. Das soll die folgende Python-Übung demonstrieren.

Eine Endlosschleife in Python (Infinite.py)

```
while True:
    eingabe = input("Geben Sie etwas ein:\n")
    print("Ihre Eingabe", eingabe, sep=": ")

    if eingabe == "a":
        print("Ihre Eingaben werden beendet.")
        break;
```

- ✓ Wegen der Anweisung `while True:` wird die Bedingung der Schleife immer wahr sein. Deshalb kann die Schleife nur aus dem Inneren heraus verlassen, wenn man nicht das Programm von außen beendet.
- ✓ Dazu finden Sie die Anweisung `break`, die bedingt ausgelöst wird. Wenn der Anwender den Kleinbuchstaben `a` eingibt, wird die Endlosschleife beendet.

## 7.15 Übungen

### Übung 1: Verzweigung

Übungsdatei: --

Ergebnisdateien: *uebung07.pdf, Maximum.py*

1. Erstellen Sie ein Struktogramm, um die Zahlen 543 und 246 zu vergleichen und die größere von beiden auszugeben.
2. Setzen Sie das Struktogramm in Python-Code um.

### Übung 2: Geschachtelte Verzweigung

Übungsdatei: --

Ergebnisdateien: *uebung07.pdf, Anrede.py*

1. Für eine Nachricht soll von einem Programm automatisch die Anrede formuliert werden. Folgende Variablen existieren:

- ✓ name: Name
- ✓ sex: Geschlecht
- ✓ currentHour: Uhrzeit (Stundenangabe)

Die Anrede soll je nach Tageszeit mit „Guten Morgen“ (0–9 Uhr), „Guten Tag“ (10–17), „Guten Abend“ (18–0 Uhr) beginnen und anschließend mit „Herr xxx“ bzw. „Frau xxx“ fortgesetzt werden. Für xxx soll der entsprechende Name eingesetzt werden.

Schreiben Sie ein entsprechendes Struktogramm.

2. Erstellen Sie aus dem Struktogramm ein Python-Programm. Für Zeichenketten stellt Python den Datentyp String zur Verfügung. Der Name und das Geschlecht sollen durch Variablen vorgegeben werden.

### Übung 3: Benutzereingaben überprüfen

Übungsdatei: --

Ergebnisdateien: *uebung07.pdf, Zugang.py*

1. In Python gibt es die Built-in Funktion `input()`, um eine Benutzereingabe von der Konsole entgegenzunehmen (`input(„Geben Sie den Usernamen ein\n“)`). Dieser kann in einer Variablen gespeichert werden. Nehmen Sie auf diese Weise einen Benutzernamen und ein Passwort entgegen.
2. Vergleichen Sie den Benutzernamen mit dem Wert „gast“ und das Passwort mit dem Wert „geheim“.
3. Wenn die Eingaben in Kombination übereinstimmen, zeigen Sie die Meldung „Zugang erlaubt“ an.
4. Wenn die Eingaben nicht stimmen, zeigen Sie die Meldung „Zugang verboten“ an.

### Übung 4: Gerade Zahlen in der Konsole anzeigen

Übungsdatei: --

Ergebnisdateien: *Geradezahlen1.py, Geradezahlen2.py, Geradezahlen3.py, uebung07.pdf*

1. Geben Sie in der Konsole die geraden Zahlen von 0 bis 100 (exklusive) aus.
2. Überlegen Sie sich mehrere Wege zur Umsetzung.

### Übung 5: Kopfgesteuerte bedingte Schleife

Übungsdatei: --

Ergebnisdateien: *uebung07.pdf, DivideBy5.py*

1. Schreiben Sie ein Programm, das zu einer eingelesenen Integer-Zahl ermittelt, wie oft mit dieser Zahl eine Division durch 5 durchgeführt werden kann. Die ermittelte Anzahl wird ausgegeben. Initialisieren Sie die einzulesende Integer-Zahl wie folgt:

```
number = int(input("Zahl\n"));
```

2. Beim Einlesen können Sie die Funktion `input()` verwenden, die einen Text über die Standardeingabe einliest. Diese Eingabe muss aber vor einer mathematischen Berechnung in eine Zahl konvertiert werden. Deshalb verwendet man in Python die Konvertierungsfunktion `int()`.
3. Zum Test, ob eine Zahl durch 5 geteilt werden kann, kann man das Moduloverfahren verwenden.
4. Beachten Sie, dass Python bei der Division eine Gleitkommazahl erzeugt, wenn sich die Operanden nicht ganzzahlig dividieren lassen. Sie müssen deshalb ggf. mit `int()` eine ganze Zahl erzwingen.

## Übung 6: Verzweigungen und Schleifen

Übungsdatei: --

Ergebnisdateien: *uebung07.pdf, Days.py*

1. Erstellen Sie ein Programm, das mit dem gegebenen letzten Tag des Vorjahres (Eingabe über die Konsole) alle Sonntage ausgibt, die auf den ersten Tag eines Monats fallen. Verwenden Sie Zahlen von 1 (Montag) bis 7 (Sonntag) für die Tage (auch als Eingabewert für den letzten Tag des Vorjahres) und 1 (Januar) bis 12 (Dezember) für die Monate.

Sie können nach diesem Verfahren vorgehen, aber auch eigene Ansätze versuchen:

- ✓ Lassen Sie den Anwender den letzten Tag des Vorjahres eingeben. Achten Sie auf den Datentyp.
- ✓ Mit einer äußeren Schleife iterieren Sie über alle Monate des Jahres. Die Werte 1 ... 12 stehen für Januar bis Dezember.
- ✓ Da die Monate unterschiedlich viele Tage haben, müssen Sie im Inneren der Schleife pro Monat diese Anzahl ermitteln. Dabei sind Schaltjahre zu berücksichtigen. Das können Sie aber rein formal implementieren und den Wert 1 für ein Schaltjahr und 0 für ein sonstiges Jahr vorgeben. Sie geben ein, ob es sich um ein Schaltjahr handeln soll oder nicht. Diese Eingabe wird aber nicht verifiziert, und die Verantwortung auf Richtigkeit liegt beim Anwender.
- ✓ Eine innere Schleife überprüft, ob ein Tag der 1. des Monats und gleichzeitig Sonntag ist. Dazu kann das Moduloverfahren zum Einsatz kommen.

Das ist die Ausgabe des Beispiels:

Queue nach Fertigstellung: [2, 3, 5, 7, 11]

2

Queue: [3, 5, 7, 11]

3

Queue: [5, 7, 11]

5

Queue: [7, 11]

7

Queue: [11]

11

Queue: []

Wo werden Queues angewendet?

Queues werden beispielsweise zur Programmierung folgender Anwendungen eingesetzt:

- Verwaltung von sequenziell abzuarbeitenden Aufgaben (Druckaufträge, Prozessverwaltung, Tasten usw.)
- Speicherverwaltung (Paging)

## 8.10 Übungen

### Übung 1: Tupel und Listen

Übungsdatei: --

Ergebnisdateien: *uebung08.pdf*, *Searchnumbers.py*

1. Welchen Index hat das n-te Element eines Tupels oder einer Liste? Begründen Sie Ihre Aussage.
2. Schreiben Sie ein Python-Programm, um in einem Tupel `numbers = (0, 10, 12, 4, 7, 20, 21, 13)` nach einer Zahl zu suchen, die ein Anwender eingibt. Das Ergebnis der Suche soll mit der Position ausgegeben werden, an der sich die Zahl befindet. Falls die Zahl nicht vorhanden ist, soll auch dazu eine Meldung ausgegeben werden. Die Anzahl der Elemente in einem Tupel erhalten Sie in Python mit der Built-in-Funktion `len()`.

### Übung 2: Tupel und Listen

Übungsdatei: --

Ergebnisdateien: *uebung08.pdf*, *FindPrimes.py*

1. Primzahlen sind Zahlen aus dem Zahlenbereich der natürlichen Zahlen größer oder gleich 2, die außer sich selbst und der Zahl 1 keine weiteren Teiler besitzen. Alle natürlichen Zahlen lassen sich in Faktoren von Primzahlen zerlegen. Erstellen Sie einen Algorithmus, der die Primzahlen von 2 bis n mithilfe des Siebs des Eratosthenes ermittelt. Bei dieser Methode werden alle Zahlen gestrichen, die sich aus anderen Zahlen zusammensetzen.
  - ✓ Schreiben Sie die Zahlen von 1 bis zur gewünschten Zahl der Größe nach geordnet in eine Liste.
  - ✓ Sie starten mit der kleinsten natürlichen Primzahl, der Zahl 2.
  - ✓ Setzen Sie alle Listeninhalte, die ein Vielfaches der Zahl 2 sind, auf 0.
  - ✓ Ist die gesamte Liste durchlaufen, wird von der Zahl 2 aus die nächstgrößere Zahl gesucht (3).
  - ✓ Alle Listeninhalte, die ein Vielfaches dieser Zahl sind, werden anschließend auf 0 gesetzt.

- ✓ Führen Sie diese Schritte so lange durch, bis das Quadrat der aktuellen Zahl größer als die letzte Zahl der Liste ist.
- ✓ In allen Listenelementen größer 1, deren Inhalt nicht auf 0 gesetzt wurde, befinden sich die gesuchten Primzahlen.

Siebzahl 2: alle Vielfache von 2 entfallen

|     |   |   |              |   |              |   |              |              |               |    |               |    |               |    |               |    |               |    |               |    |               |    |               |    |     |
|-----|---|---|--------------|---|--------------|---|--------------|--------------|---------------|----|---------------|----|---------------|----|---------------|----|---------------|----|---------------|----|---------------|----|---------------|----|-----|
| alt | 2 | 3 | 4            | 5 | 6            | 7 | 8            | 9            | 10            | 11 | 12            | 13 | 14            | 15 | 16            | 17 | 18            | 19 | 20            | 21 | 22            | 23 | 24            | 25 | ... |
| neu | 2 | 3 | <del>4</del> | 5 | <del>6</del> | 7 | <del>8</del> | <del>9</del> | <del>10</del> | 11 | <del>12</del> | 13 | <del>14</del> | 15 | <del>16</del> | 17 | <del>18</del> | 19 | <del>20</del> | 21 | <del>22</del> | 23 | <del>24</del> | 25 | ... |

Siebzahl 3: zusätzlich entfallen alle Vielfache von 3

|     |   |              |              |   |              |   |              |              |               |    |               |    |               |               |               |    |               |    |               |               |               |    |               |    |     |
|-----|---|--------------|--------------|---|--------------|---|--------------|--------------|---------------|----|---------------|----|---------------|---------------|---------------|----|---------------|----|---------------|---------------|---------------|----|---------------|----|-----|
| alt | 2 | 3            | <del>4</del> | 5 | <del>6</del> | 7 | <del>8</del> | <del>9</del> | <del>10</del> | 11 | <del>12</del> | 13 | <del>14</del> | 15            | <del>16</del> | 17 | <del>18</del> | 19 | <del>20</del> | <del>21</del> | <del>22</del> | 23 | <del>24</del> | 25 | ... |
| neu | 2 | <del>3</del> | <del>4</del> | 5 | <del>6</del> | 7 | <del>8</del> | <del>9</del> | <del>10</del> | 11 | <del>12</del> | 13 | <del>14</del> | <del>15</del> | 16            | 17 | <del>18</del> | 19 | <del>20</del> | <del>21</del> | 22            | 23 | <del>24</del> | 25 | ... |

usw.

2. Setzen Sie den Algorithmus aus Aufgabe 1 in Python-Code um. Verwenden Sie zum Einlesen der Zahl, bis zu der die Primzahlen ermittelt werden sollen, die Funktion `input()`. Nutzen Sie die `append()`-Methode zum Aufbau der Liste. Speichern Sie das Programm unter *FindPrimes.py*.

### Übung 3: Listen

Übungsdatei: --

Ergebnisdateien: *Warenkorb1.py*

1. Erstellen Sie auf Basis einer Liste ein Python-Programm für einen Warenkorb.
2. Die Anwendung soll in einer Schleife ein Produkt als freie Texteingabe angeben (mit der `input()`-Funktion).
3. Das Produkt wird mit der `append()`-Methode der Liste (dem Warenkorb) hinzugefügt.
4. Lassen Sie den Anwender entscheiden, wie daran weiter vorgegangen wird:
  - ✓ weiter einkaufen
  - ✓ bestellen oder
  - ✓ die Bestellung abbuchen

### Übung 4: Dictionaries

Übungsdatei: --

Ergebnisdateien: *Warenkorb2.py*

1. Erstellen Sie auf Basis eines Dictionaries ein Python-Programm für einen Warenkorb.
2. Die Anwendung soll in einer Schleife ein Produkt als freie Texteingabe angeben (mit der `input()`-Funktion).
3. Dem Produkt soll eine Anzahl zugeordnet werden, die ebenso als freie Texteingabe mit der `input()`-Funktion eingegeben wird.
4. Produkt und Anzahl werden mit der `update()`-Methode dem Dictionary (dem Warenkorb) hinzugefügt.
5. Lassen Sie den Anwender entscheiden, wie daran weiter vorgegangen wird:
  - ✓ weiter einkaufen
  - ✓ bestellen oder
  - ✓ die Bestellung abbuchen
6. Geben Sie zu jeder Entscheidung des Anwenders eine passende Meldung in der Konsole aus.

- ① Zum Generieren einer zufälligen Zahl in Python wird das Modul `random` verwendet. Dieses wird mit der `import`-Anweisung eingebunden.
- ② Dann wird der Zufallsgenerator mit `random.seed()` initialisiert.
- ⑦ Die Funktion `randint()` wird mit zwei Parametern aufgerufen und liefert eine zufällige Ganzzahl aus dem durch die Parameter definierten Bereich.
- ④ Der Zufallswert wird ausgegeben.

## 9.8 Übungen

### Übung 1: Funktionen, Prozeduren und Methoden ohne Rückgabewert

Übungsdatei: --

Ergebnisdateien: *uebung09.pdf, MoveRight.py*

1. Schreiben Sie eine Prozedur in Pseudocode, die alle Elemente eines Feldes um jeweils eine Position nach rechts verschiebt. Das letzte Element soll das erste Element werden.
2. Setzen Sie den Pseudocode aus Aufgabe 1 in Python-Code um, wobei Sie eine Funktion ohne Rückgabewert erstellen.
3. Erstellen Sie eine Prozedur in Pseudocode, der als Parameter der Radius einer Kugel übergeben wird. In der Prozedur sind der Umfang der Kugel, die Mantelfläche und das Volumen zu berechnen. Die Werte sollen anschließend dem aufrufenden Programm zur Verfügung stehen.

Umfang einer Kugel (Kreis):  $2 \cdot \pi \cdot R$        $\pi$  entspricht der Konstanten  $\pi = 3.14$

Mantelfläche der Kugel:  $4 \cdot \pi \cdot R^2$        $R$  entspricht dem Radius

Volumen einer Kugel:  $4 / 3 \cdot \pi \cdot R^3$

### Übung 2: Funktionen und Methoden mit Rückgabewert

Übungsdatei: --

Ergebnisdateien: *uebung09.pdf, FindMin.py, UsePower.py*

1. Schreiben Sie eine Funktion in Pseudocode, die das Minimum dreier verschiedener Werte bestimmt und an das Hauptprogramm zurückgibt.
2. Lösen Sie die Aufgabe 1 mit einem Python-Programm. Speichern Sie das Programm unter *FindMin.py*.
3. Erstellen Sie eine Funktion in Pseudocode, die die n-te Potenz einer Zahl  $x$  berechnet ( $y = x^n$ ). Die Zahlen  $x$  und  $n$  sollen der Funktion als Parameter übergeben werden, wobei  $x$  und  $n$  natürliche Zahlen (0, 1, 2, 3 ...) sind. Das Ergebnis soll im Hauptprogramm ausgegeben werden.
4. Erstellen Sie für die Aufgabe 3 ein Python-Programm unter Verwendung passender Methoden. Die Methode soll mit Eingaben des Anwenders für die Basis und den Exponenten getestet werden. Speichern Sie das Programm unter *UsePower.py*.

## 10.6 Übung

## Iterative und rekursive Algorithmen

Übungsdatei: --

Ergebnisdateien: *uebung10.pdf*, *AddRec.py*, *AddIter.py*,  
*KingChess.java*,  
*GreatestCommonDivisor.java*

1. Die Multiplikation der ganzen Zahlen  $x$  und  $y$  kann auch als  $x$ -maliges Addieren der Zahl  $y$  gelöst werden. Zum Beispiel ist  $3 * 4 = 4 + 4 + 4 = 12$ .  
Erstellen Sie eine rekursive Funktion für diese Operation.
2. Lösen Sie die Übung 1 mithilfe eines iterativen Algorithmus.
3. Erstellen Sie für die Aufgaben 1 und 2 jeweils ein Python-Programm. Speichern Sie die Programme unter *AddRec.py* und *AddIter.py*.
4. Berechnen Sie mit einem rekursiven Algorithmus die Anzahl der Weizenkörner, ausgehend von einem Schachspiel. Das erste Feld soll mit einem Korn belegt sein, das zweite Feld mit zwei Körnern und so fort. Die Zahl der Körner soll auf jedem Feld verdoppelt werden.
5. Lösen Sie die Aufgabe 4 mit einem Python-Programm. Speichern Sie das Programm unter *KingChess.py*.
6. Erstellen Sie einen iterativen Algorithmus, der den größten gemeinsamen Teiler von zwei vorgegebenen ganzen Zahlen ermittelt. Verwenden Sie dazu den Algorithmus von Euklid, der die folgenden Schritte umfasst.

- ✓ Ausgangspunkt sind zwei ganze Zahlen, z. B. 75 und 54.
- ✓ Bilden Sie durch die ganzzahlige Division den Quotienten aus der größeren Zahl (Dividend) und der kleineren Zahl (Divisor), z. B.  $75/54$ .
- ✓ Ist der Rest der ganzzahligen Division verschieden von 0, wird ein neues Zahlenpaar aus der kleineren Zahl und dem Rest der ganzzahligen Division gebildet, z. B. 54 und 21.
- ✓ Bilden Sie davon analog zum ersten Zahlenpaar den Quotienten, z. B.  $54/21$ .
- ✓ Wiederholen Sie den Vorgang so lange, bis der Rest der ganzzahligen Division 0 ist.
- ✓ Der größte gemeinsame Teiler ist der Divisor der letzten Division.

| Zahlenpaar |    | Quotient | Rest |   |
|------------|----|----------|------|---|
| 75         | 54 | 1        | 21   | : |
| 54         | 21 | 2        | 12   | : |
| 21         | 12 | 1        | 9    | : |
| 12         | 9  | 1        | 3    | : |
| 9          | 3  | 3        | 0    | : |

größerer gemeinsamer T

7. Lösen Sie die Aufgabe 6 mit einem Python-Programm. Speichern Sie das Programm unter *GreatestCommonDivisor.py*.
8. Die Verzeichnisse und Dateien in einem Betriebssystem sind in der Regel innerhalb einer Baumstruktur angeordnet. Um eine bestimmte Datei zu suchen, werden die Ordner und Unterordner nacheinander durchsucht, bis die Datei gefunden wird. Wie kann ein Algorithmus zum Suchen einer Datei in einer solchen Baumstruktur aussehen? Geben Sie eine iterative und eine rekursive Lösung an.

## 11.5 Schnellübersicht

| Was bedeutet ...? |                                                                                           |
|-------------------|-------------------------------------------------------------------------------------------|
| Event             | Ein Ereignis, auf das reagiert werden kann                                                |
| Scheduler         | Eine zeitgesteuerte Aufgabe                                                               |
| Event Handler     | Ein Reaktionsmechanismus auf ein Ereignis                                                 |
| Bubble-Phase      | Das Weiterreichen eines Eventobjekts in einem verschachtelten Baum an möglichen Listener. |

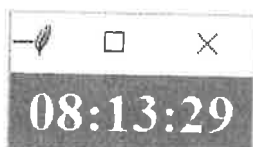
## 11.6 Übung

Fragen zur Ereignisbehandlung

Übungsdatei: --

Ergebnisdateien: *uebung11.pdf, MyClockGUI.py*

1. Erstellen Sie als Python-Programm eine Uhr mit einer grafischen Oberfläche.
2. Verwenden Sie das `time`- und das `tkinter`-Modul.
3. Deklarieren Sie eine Funktion, in der die Zeit ermittelt wird. Es gibt verschiedene Möglichkeiten, um in Python ein Datumsobjekt für einen beliebigen Zeitpunkt zu erstellen. In der Dokumentation finden Sie entsprechende Informationen (<https://docs.python.org/3/library/time.html>). Verwenden Sie die Methode `strptime()` aus dem Modul `time`. Diese nimmt sogenannte Direktiven zur Aufbereitung des Datumsobjekts an. Mit `%H` erhalten Sie die Stunden im 24-Stunden-Format, mit `%M` die Minuten und mit `%S` die Sekunden.
4. Wenn das Programm startet, soll die Oberfläche ein Label anzeigen, in dem die aktuelle Uhrzeit zu sehen ist.
5. Verwenden Sie den `pack`-Geometry-Manager (<https://docs.python.org/3/library/tk.html>). Damit wird die Fenstergröße auf den Raum optimiert, den das Label benötigt.



So soll die grafische Uhr aussehen